LAB #13

Adv. CiC
Spring 2026

Agenda

1. Housekeeping
2. Algorithms
3. Big O Notation
4. Parallelization
5. Lab 12 Exercise Re-Do!

Housekeeping

Due this past week

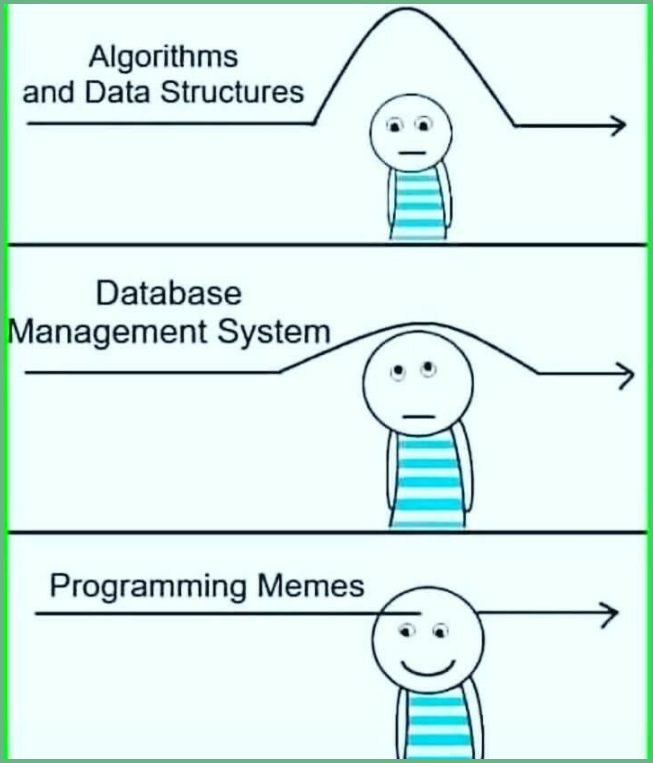
- Project, pt. 6
- Reading 13

Upcoming

1. Part 7 - **4/23 @ 3pm**
2. Reading 14 - **4/23 @ 3pm**

Announcements

- Aidan's performance
- Grading catch up - by this weekend (promise!)
- **Final regrades due date - 4/30**



Questions

(on Lectures, Labs, or Readings)?

Algorithms

What is an algorithm?

set of step-by-step instructions (logic) to solve a problem or complete a task

Algorithm for Making Tea

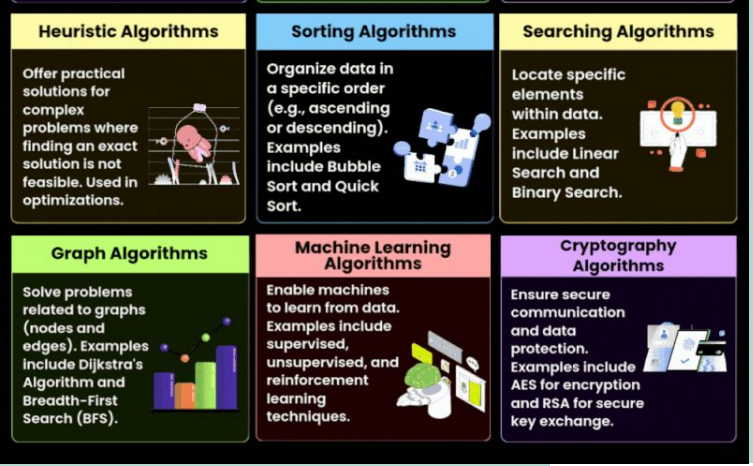
Step 01 : Boil water in a Saucepan.

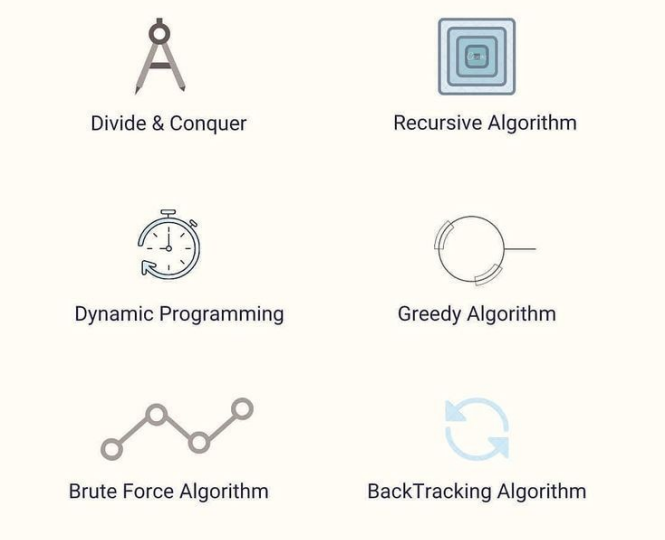
Step 02 : Add Sugar and Tea Powder and boil it for 3-4 minutes.

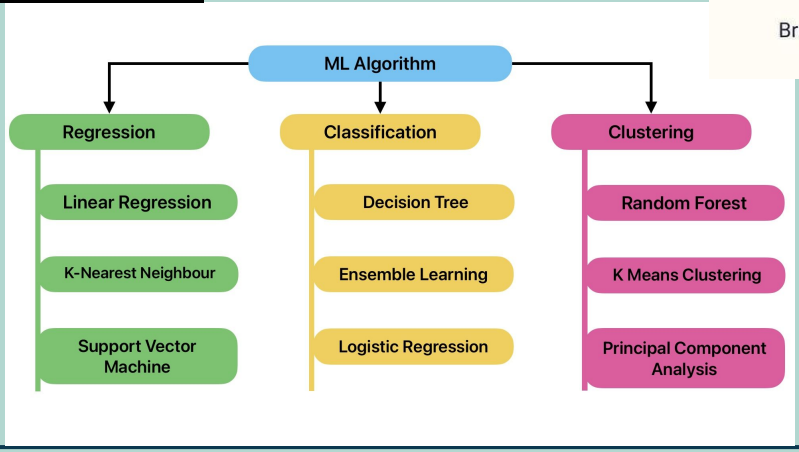
Step 03 : Add some Milk and boil it for 6-7 minutes.

Step 04 : Eventually the color will change to brown.

Step 05 : Strain the tea in a cup and Serve it.

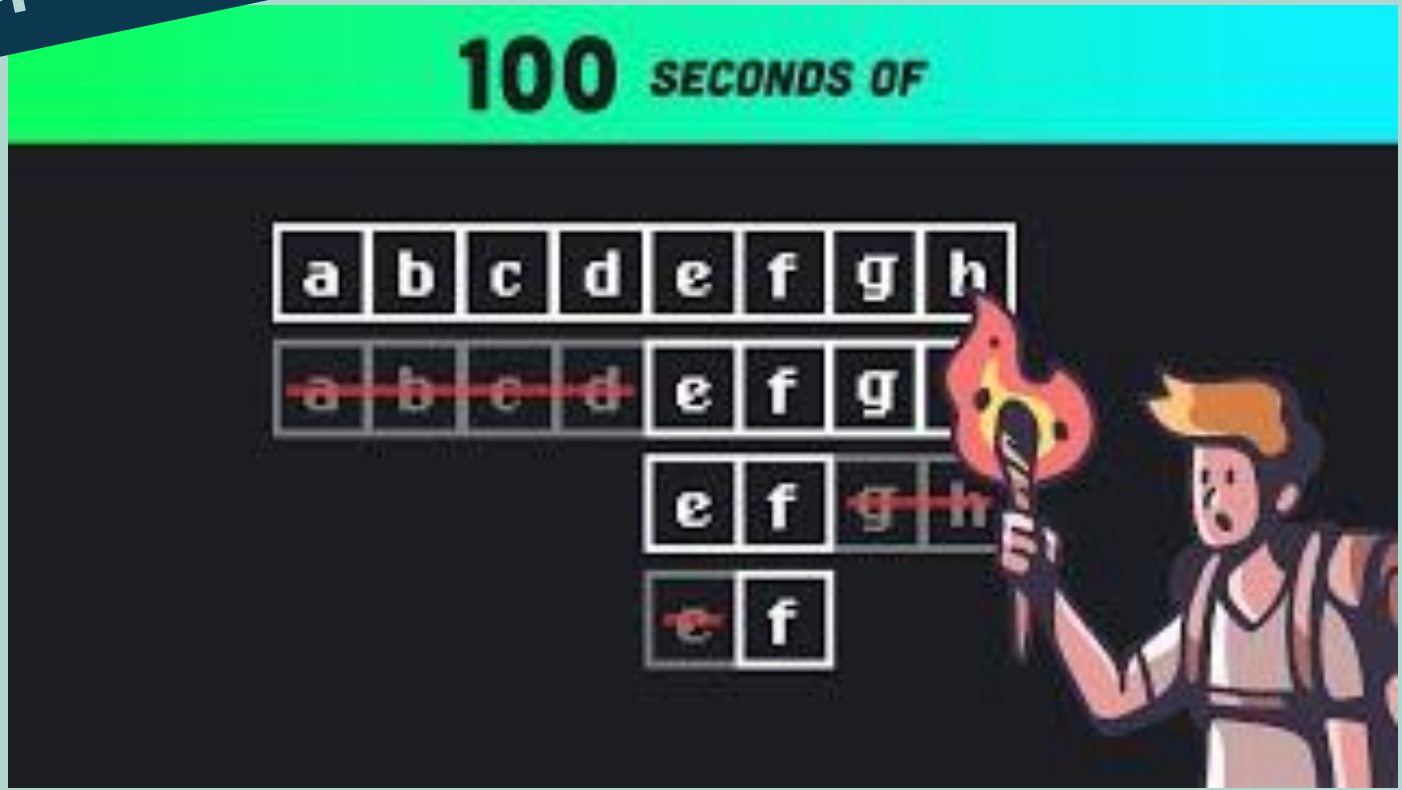




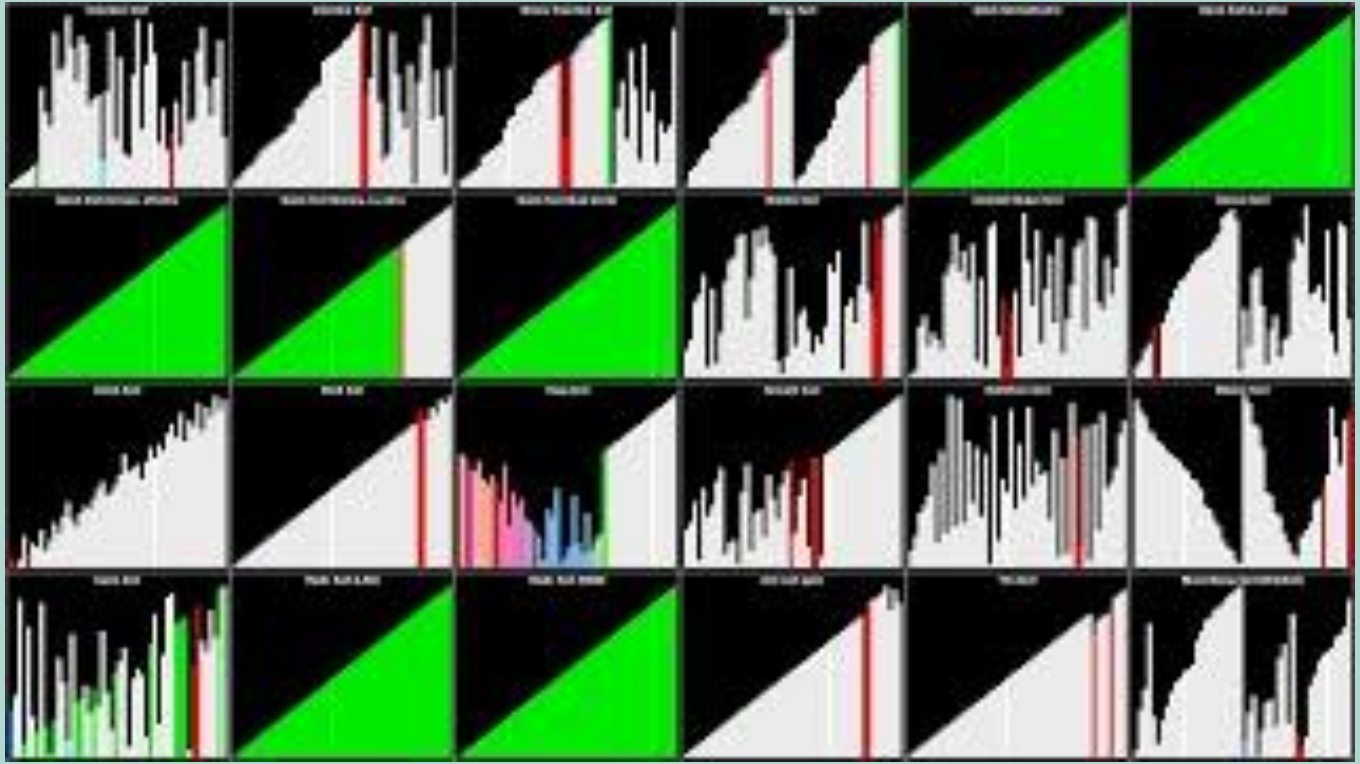


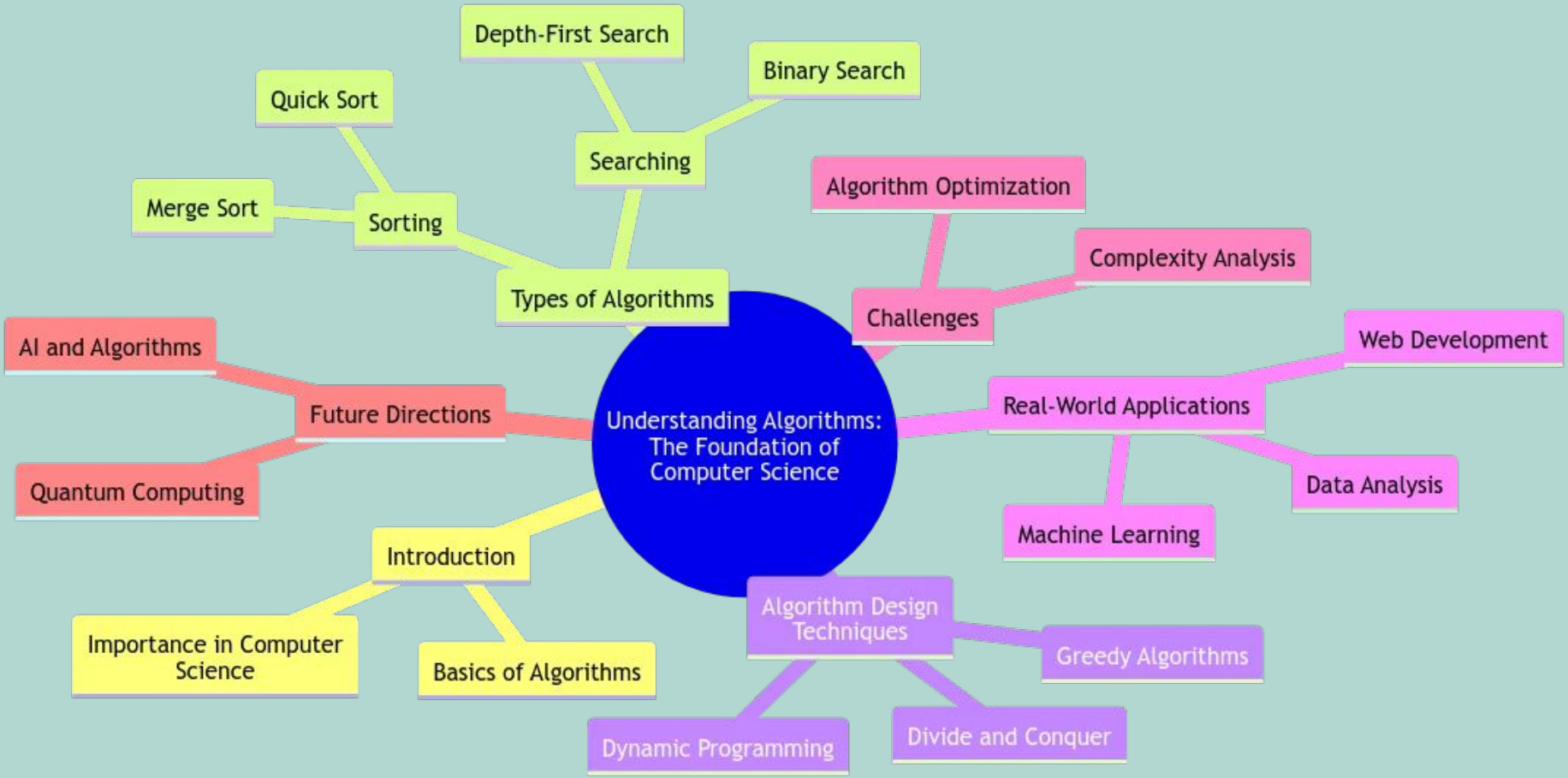
Brute Force: solving a problem by trying every possible option, one by one, until it finds the answer

Example: Search



Example: Sorting




Understanding Algorithms: The Foundation of Computer Science
Types of Algorithms
Searching
Depth-First Search
Binary Search
Sorting
Quick Sort
Merge Sort
Challenges
Algorithm Optimization
Complexity Analysis
Real-World Applications
Web Development
Data Analysis
Machine Learning
Future Directions
AI and Algorithms
Quantum Computing
Introduction
Importance in Computer Science
Basics of Algorithms
Algorithm Design Techniques
Greedy Algorithms
Divide and Conquer
Dynamic Programming

Big O

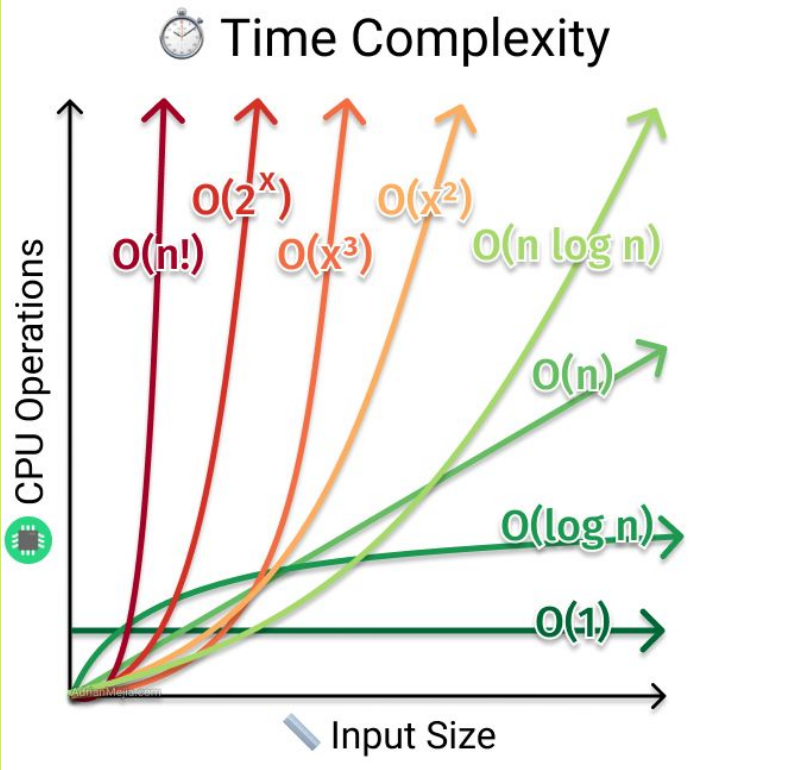
How do we evaluate algorithms?

Time & Space Complexity

1. **Time complexity** measures how an algorithm's runtime grows with input size

2. **Space complexity** measures how much memory an algorithm needs as the input grows larger

3. **Typical trade-off: time vs. memory**



Example: Sorting

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Quick Sort	O(nlogn)	O(nlogn)	O(n^2)	O(n)
Heap Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(1)
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n + k)
Bucket Sort	O(n + k)	O(n + k)	O(n^2)	O(n)

Why would you ever choose a “slower” algorithm?

1. **Memory is limited** — if your system has strict memory constraints, a slower algorithm that uses less space may be the only viable option

2. **You know your data** — if you can predict the input (e.g. nearly sorted data), a "slower" algorithm may actually perform closer to its best case than its worst

3. **Simplicity matters** — a slower but readable algorithm is easier to maintain, audit, and explain to others, which matters in policy and governance contexts

4. **Accuracy over speed** — some faster algorithms are approximations; a slower algorithm may guarantee an exact, correct answer when precision is critical

12:06:57 0.07 K/s 9%

Thread

Sunjay liked

jwcarroll
@jwcarroll

Alternative Big O notation:

O(1) = O(yeah)
O(log n) = O(nice)
O(n) = O(ok)
$O(n^2)$ = O(my)
$O(2^n)$ = O(no)
O(n!) = O(mg!)

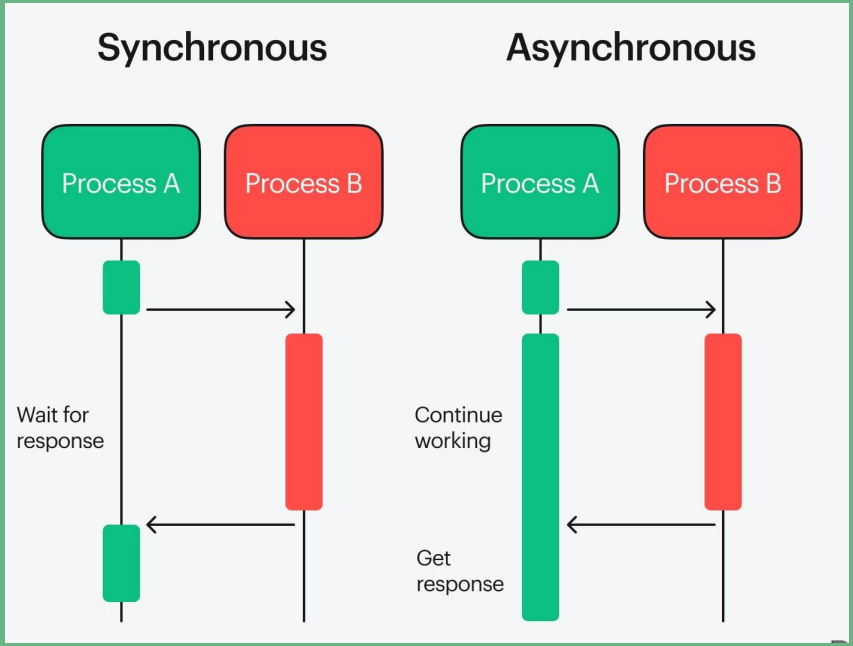
8:10 PM · 06 Apr 19 · Twitter for Android

3,665 Retweets **9,265** Likes

jwcarroll @jwcarroll · 8h

Tweet your reply

Parallelization



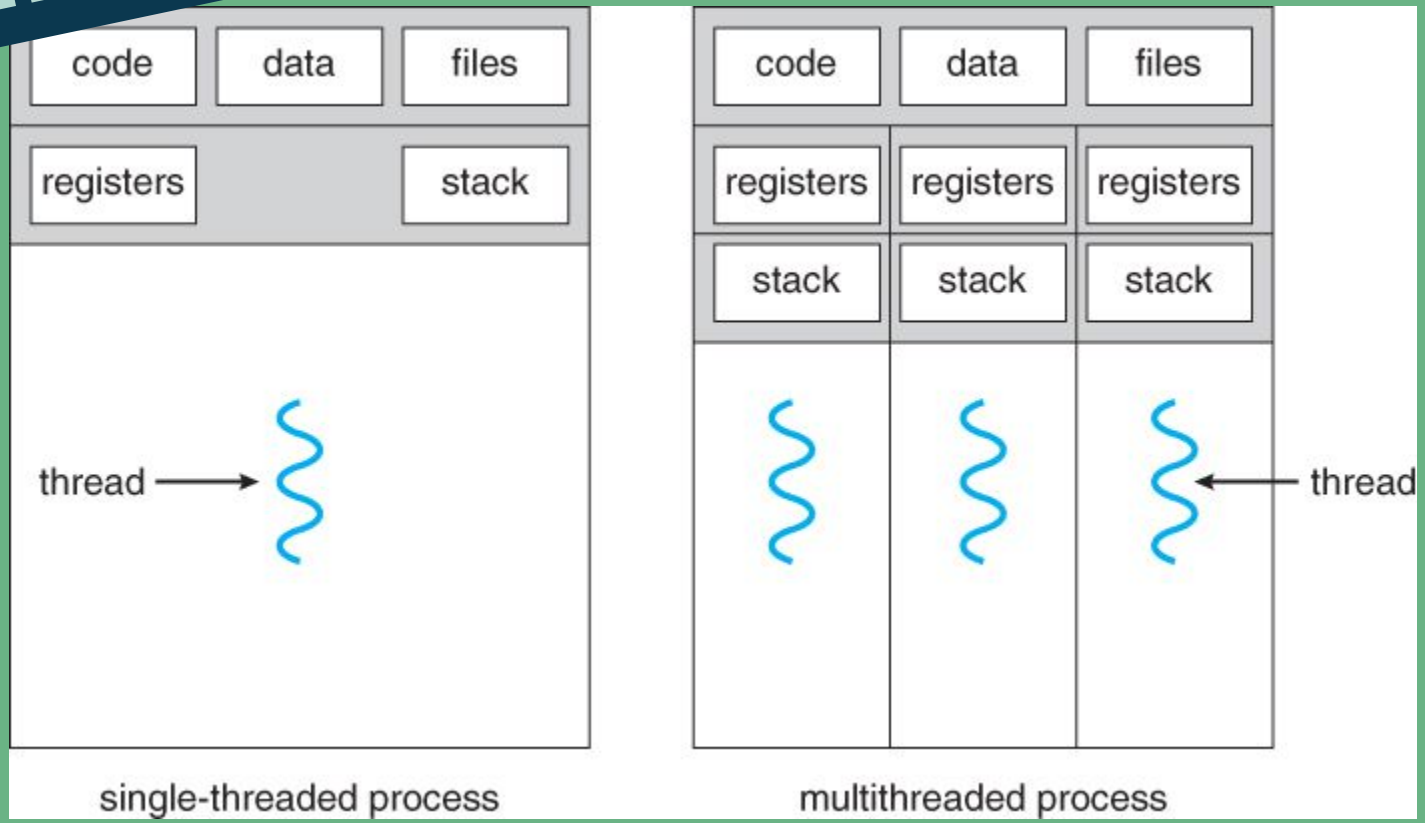
Sync

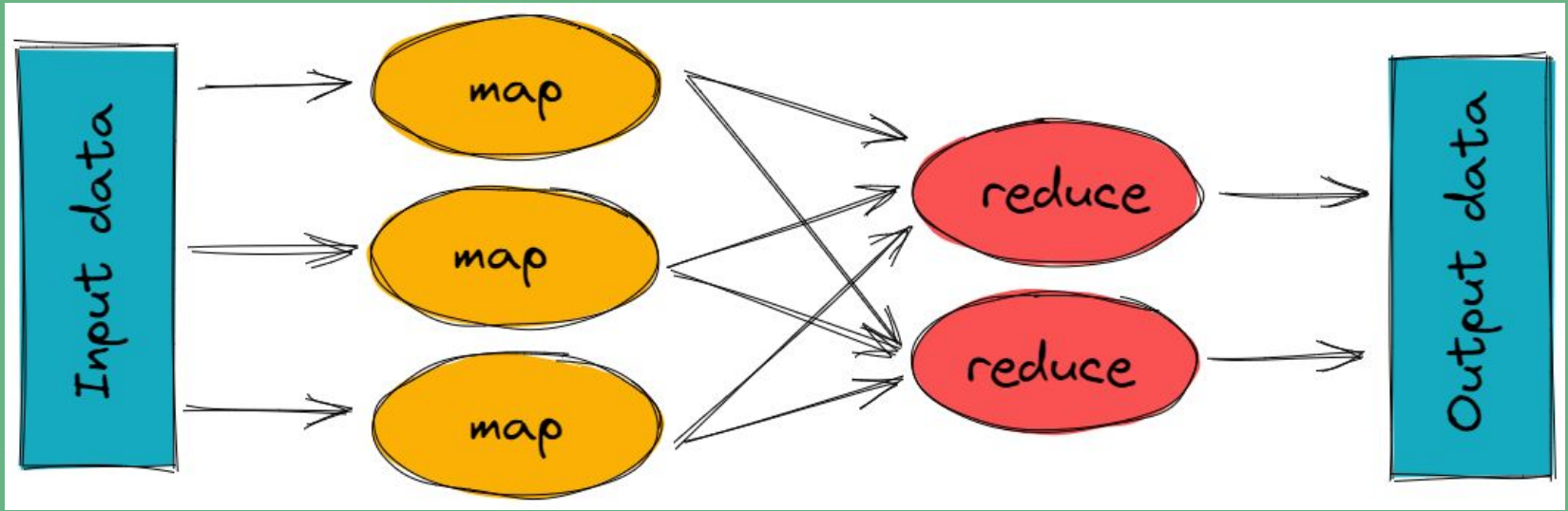
- **Tasks run one at a time, in order** — each must finish before the next begins
- Simple to read and debug, but can be slow if a task takes a long time

Async

- **Tasks can run concurrently** — the program doesn't wait for one to finish before starting another
- More efficient for things like network requests, but harder to implement & reason about

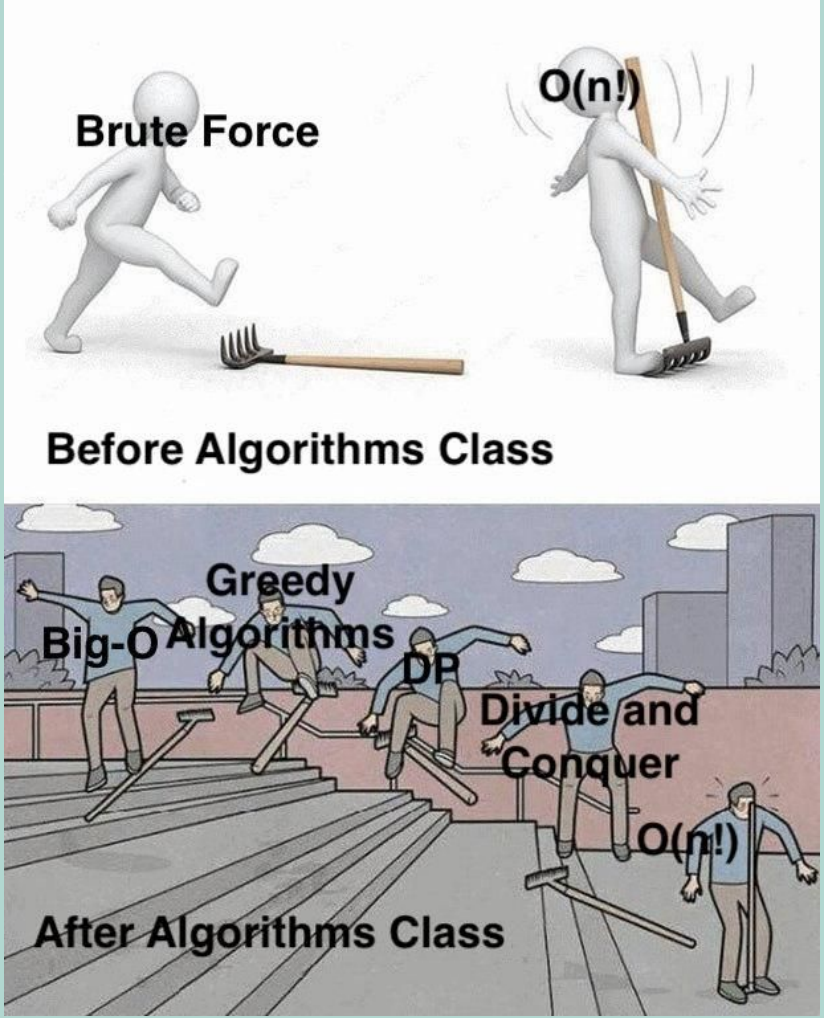
Threads


code
data
files
registers
stack
thread
single-threaded process
code
data
files
registers
registers
registers
stack
stack
stack
thread
multithreaded process



MapReduce

- **Map:** break a large problem into smaller chunks and process them in parallel across many “workers”
- **Reduce:** combines all the results from the map step into a single final output


Brute Force
O(n!)
Before Algorithms Class
Greedy
Big-O
Algorithms
DP
Divide and
Conquer
O(n!)
After Algorithms Class

[redacted]

3 hrs

CS Majors be like

Bae: Come over
Dijkstra: But there are so many routes to take and
I don't know which one's the fastest
Bae: My parents aren't home
Dijkstra:

Dijkstra's algorithm

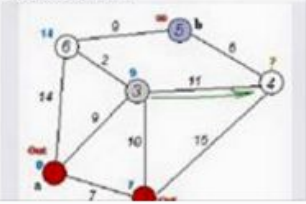
Graph search algorithm

Not to be confused with Dykstra's projection algorithm.

Dijkstra's algorithm is an algorithm for finding the shortest paths between nodes in a graph, which may represent, for example, road networks. It was conceived by computer scientist Edsger W. Dijkstra in 1956 and published three years later.[1][2]

The algorithm exists in many variants; Dijkstra's original variant found the shortest path between two nodes,[2] but a more common variant fixes a single node as the "source" node and finds shortest paths from the source to all other nodes in the graph, producing a shortest-path tree.

Dijkstra's algorithm



Like Comment

237

EXERCISE

Lfg